Crash Twin - diagrams

Notes for crash_twin_bpmn.drawio (4 pages)

WIUT Hackathon 2026 - Team UsedByAI
Prepared 10 October 2026

How to open the file

Open crash_twin_bpmn.drawio in app.diagrams.net (File > Open from > Device) or in the draw.io desktop app. It has four pages, listed as tabs at the bottom. Every shape can be edited.

Each page was drawn with your bpmn-architect skill and passes its validator (Gate 1: 0 failures). I also rendered each page and checked it by eye (Gate 2). The Python sources are kept, so any page can be regenerated after an edit.

Page 1 - Business flow (minor collision to settled claim)

This shows the process after Crash Twin is in place. It is split into two pools, joined by link event A.

At the scene (Driver, Crash Twin, Traffic police)

- After a minor collision with no injuries, a driver opens the Crash Twin link and films a 60-second walk-around plus close-ups of the damage.
- Crash Twin checks the capture and blurs plates and faces. If the capture is bad, the driver is asked to retake it (red loop).
- Crash Twin then builds the 3D twin, measures the scene, applies the traffic rules and sends a draft report with a 3D replay link.
- If both drivers agree on who is at fault, they move the cars and clear the lane straight away. If they do not agree, a traffic inspector reviews the twin and decides.
- The fault decision always stays with the police or a court. Crash Twin only prepares the evidence.

Insurance claim (Driver, Crash Twin, Body shop, Insurer)

- The driver files a claim with the case link, and the insurer registers it.
- Crash Twin runs a consistency and fraud check: does the damage match the scene?
- If it matches, the repair estimate is approved quickly. If it does not, a field adjuster inspects the car, and the claim is either approved or refused.
- The body shop takes the car in using the 3D damage map, repairs it and invoices the insurer. The insurer pays and closes the claim.

Page 2 - Technical pipeline (one case, from video to verdict)

Capture and upload

- The phone web app records video, adds GPS and uploads it.
- The API creates the case and puts a job on the GPU queue.

GPU worker

- Samples frames, checks their quality and blurs plates and faces. If there are not enough sharp frames, a retake hint goes back to the phone.
- VGGT-1B-Commercial computes camera poses and a point cloud. COLMAP is the fallback.
- A parallel fork then runs:
  - cars: detect, segment, 3D box, make/model, metric scale;
  - damage masks lifted onto the 3D cars;
  - road context: lanes, signs, markings and OpenStreetMap;
  - a Gaussian splat for the 3D replay, in the background.

Analysis and report

- Physics estimates the impact direction and a speed range (Monte Carlo).
- The rules engine evaluates rule predicates against Resolution 172.
- If confidence is 0.6 or higher, the verdict is "likely violated / not violated". Below that, it is "cannot tell", with a list of the missing evidence.
- An LLM only quotes the clause text and writes the Uzbek/Russian report. It never decides the verdict.
- The API saves the case and renders a PDF, and the phone shows the case page.

Timing targets: instant report in about 1-3 min, full 3D replay in 5-15 min. These are estimates and must be measured.

Page 3 - Data, training and testing

Three data sources run in parallel:

1. Public datasets: VehiDE, CarDD, Nexar, NHTSA CISS. Check each licence and list every source in the README.
2. Our own captures: parked cars and staged toy-car crashes, labelled in CVAT with tape-measured ground truth.
3. Synthetic data: Tashkent OpenStreetMap junctions imported into CARLA, with scripted crashes at known speeds and known rule clauses.

Test sets T1-T7 are frozen BEFORE any training. No reported number may come from data the model has seen. After that we fine-tune the damage segmentation model and the car-model classifier, run the full pipeline on every test set, and loop on error analysis until the targets are met. We then publish the metrics together with the failure cases.

Page 4 - Architecture and tech stack (demo deployment)

Tier	Component	Technology
Users	Driver phone; insurer / police / jury	Any browser, no app install
Web	Capture page; case page + 3D viewer	Next.js PWA (MediaRecorder), three.js splat viewer, Vercel
Backend	API, queue, database, reports	FastAPI, Redis + RQ, PostgreSQL, PDF report service
GPU worker	M1-M8 pipeline	PyTorch; VGGT-1B-Commercial, COLMAP/GLOMAP, RT-DETR, SAM 2, gsplat, physics + rules engine
Data + external	Files, map, wording	S3 / MinIO object storage, OpenStreetMap Overpass, LLM API

The demo hardware is one rented GPU server with 24 GB or more (RTX 4090, L4 or A100 class, from RunPod, Vast.ai or Lambda). It runs both the backend and the worker. The front-end is static and deployed from the public repository.

Licences

Component	Licence
VGGT-1B-Commercial	commercial use allowed (do not use the original VGGT checkpoint)
COLMAP / GLOMAP	BSD
RT-DETR	Apache-2.0
SAM 2	Apache-2.0
gsplat	Apache-2.0
CARLA	MIT
OpenStreetMap	ODbL, attribution required
YOLO11	AGPL-3.0, so prefer RT-DETR for a product

Open questions before the final

- Real crash and claim numbers for Tashkent (for the business slide).
- Does Uzbekistan allow an "europrotocol" (no-police form for a minor crash)? Page 1 assumes the police still decide when the drivers disagree.
- How a real insurer handles a claim today (to check the as-is process against our to-be).

A bug found in the skill

In bpmn_emit.py, multi-line labels lose their line breaks. A raw newline inside an XML attribute is turned into a space when the file is parsed, so "1.1 Open link\nfilm..." shows as one long line.

Fix: escape the newline as
 when writing labels (one line in the escape call). My copy has this fix, and I also added an optional per-edge label offset (loff) for labels on very short vertical edges. Worth putting both back into the skill.